

GlobleJump

Stripe Payment Journey

End-to-end money flow: **Advisor** → **Seeker** → **Admin**

Checkout · Connect · Webhooks · Refunds · Payouts

Source of truth: `app/services/payment_service.py`, `payout_service.py`,
`app/api/v1/payments.py`, `advisors.py`, `admin.py` (PRD §3.10).

Internal engineering reference · generated from codebase

1. Actors & money split

Three platform roles participate. Stripe is the processor; the API records ledger state on **Transaction** and **PayoutRequest** rows.

Advisor	Seeker	Admin
Offers services, accepts bookings, optionally connects Stripe Express for destination charges, requests withdrawals from earned balance.	Books a consultation, pays via Stripe Checkout for a confirmed unpaid booking, receives receipt + invoice.	Issues refunds, reviews payout queue (complete/reject), monitors platform finance & analytics.

Default fee configuration (Settings)

Setting	Default	Effect
PLATFORM_COMMISSION_RATE	15%	Platform fee taken from booking price
TAX_WITHHOLDING_RATE	8%	Withheld from advisor share
PAYOUT_PROCESSING_FEE_RATE	2%	Fee on manual payout requests

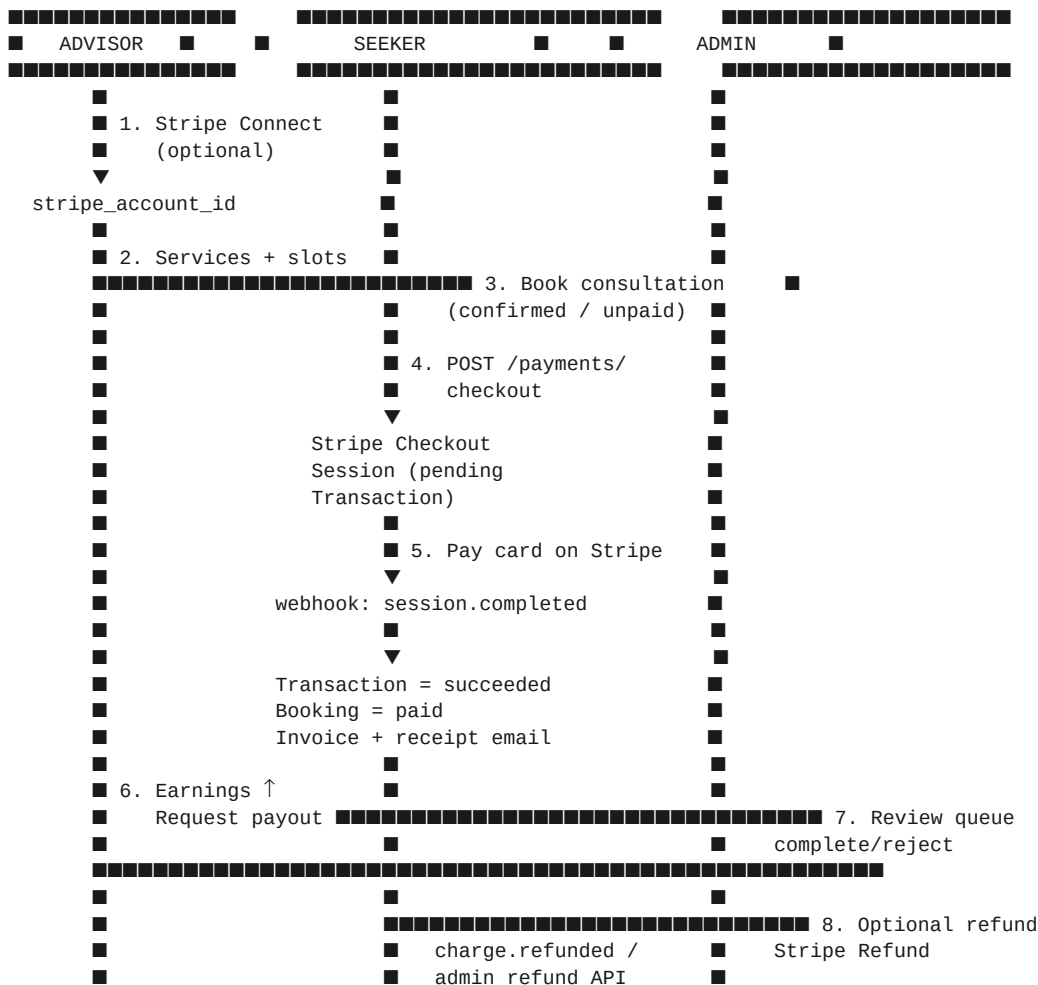
Example for a **\$100** booking: commission **\$15** + tax **\$8** → advisor payout ledger **\$77**. With Connect, Stripe `application_fee_amount` = commission+tax (cents); `transfer_data.destination` = advisor connected account.

2. Happy-path sequence (Advisor → Seeker → Admin)

Order below matches production intent. Connect is optional; without it, funds land on the platform Stripe account and advisor balance is ledger-only until admin completes a payout.

A1	Advisor	Complete Stripe Connect onboarding	POST /advisors/me/stripe-connect → Account Links; store <code>stripe_account_id</code> on <code>AdvisorProfile</code>
A2	Advisor	Publish services + availability	Advisor profile services + weekly slots so seekers can book
S1	Seeker	Create booking	POST /bookings → status confirmed, <code>payment_status</code> unpaid
S2	Seeker	Start Checkout	POST /payments/checkout { <code>booking_id</code> } → Stripe Checkout Session + Transaction pending
S3	Seeker	Pay on Stripe hosted page	Card payment on Stripe; success/cancel URLs under <code>FRONTEND_URL/bookings/{id}</code>
W1	Stripe→API	Webhook <code>checkout.session.completed</code>	POST /payments/webhook → Transaction succeeded, booking paid, invoice #, receipt email
A3	Advisor	See earnings / request payout	GET earnings; POST /advisors/me/payouts (reserves available balance)
M1	Admin	Approve or reject payout	PATCH /admin/payouts/{ <code>id</code> } complete reject — ledger only (no Stripe Transfer API today)
M2	Admin	Optional refund	POST /admin/payments/{ <code>id</code> }/refund → Stripe Refund; txn refunded / partially_refunded

3. Journey flowchart



4. Stripe objects & local records

Stripe	Local DB	Notes
Account (Connect Express)	AdvisorProfile.stripe_account_id	Created in create_connect_account
Checkout Session	Transaction.stripe_checkout_session_id	status=pending until webhook
PaymentIntent / Charge	stripe_payment_intent_id / stripe_charge_id	Filled on session.completed
Refund	refunded_* + status	Admin API and/or charge.refunded webhook
(manual bank / Connect)	PayoutRequest	Admin complete does not call Stripe Transfer today

5. Key API endpoints by role

Advisor

POST/GET /advisors/me/stripe-connect · GET earnings · GET/POST /advisors/me/payouts · GET /advisors/me/payouts/preview · GET payment detail / invoice (party access)

Seeker

POST /payments/checkout · GET /payments/config (publishable key) · GET /payments/summary · GET /payments/history · GET /payments/{id} · GET /payments/{id}/invoice?perspective=seeker

Admin

GET platform payment summary · list transactions · POST /admin/payments/{id}/refund · GET /admin/payouts · PATCH /admin/payouts/{id} (complete/reject) · finance analytics including refund_trend

System

POST /payments/webhook — verifies Stripe-Signature when STRIPE_WEBHOOK_SECRET is set; handles **checkout.session.completed**, **checkout.session.expired**, **charge.refunded**.

6. Transaction & booking state machine

Transaction.status	When	Booking.payment_status
pending	Checkout session created	unpaid
succeeded	checkout.session.completed	paid
failed	checkout.session.expired	unchanged (unpaid)
partially_refunded / refunded	Admin refund or charge.refunded	refunded

7. Guardrails (from create_checkout_session)

- Only the booking's seeker may start checkout.
- Booking must be **confirmed** and **unpaid**.
- One Transaction per booking (duplicate checkout rejected).
- If advisor has stripe_account_id → Connect destination charge + application fee; else platform collects full amount and tracks advisor_payout_usd on the Transaction.

8. Payout balance (advisor ledger)

available = SUM(advisor_payout_usd for succeeded txns) – SUM(payout amounts where status ≠ rejected). Pending requests reserve balance. Completing a payout is an admin ledger action (processed_at / processed_by); rejecting releases the reserve.

9. Dev / test notes

Script scripts/test_payment_flow.py drives register → book → checkout → synthetic webhook (dev mode skips signature when webhook secret unset). Connect can be skipped in sandbox; payments then settle to the platform account.